

Zero to Hero Study Handbook: lora-dispute-ticket-routing

Module 1: Foundations & Architecture

1.1 What this project does

This repository builds and studies an 8-class support-ticket router for banking complaints using QLoRA fine-tuning on Qwen/Qwen2.5-1.5B-Instruct.

Primary use cases in this codebase: - Train an adapter that routes complaint text into one of 8 operational queues. - Compare zero-shot vs fine-tuned routing quality. - Run inference for one complaint or a batch with top-1 confidence and top-3 suggestions. - Analyze calibration and confusion patterns for production-style triage decisions.

1.2 Core paradigms and patterns used here

Definition first, then where it appears in code:

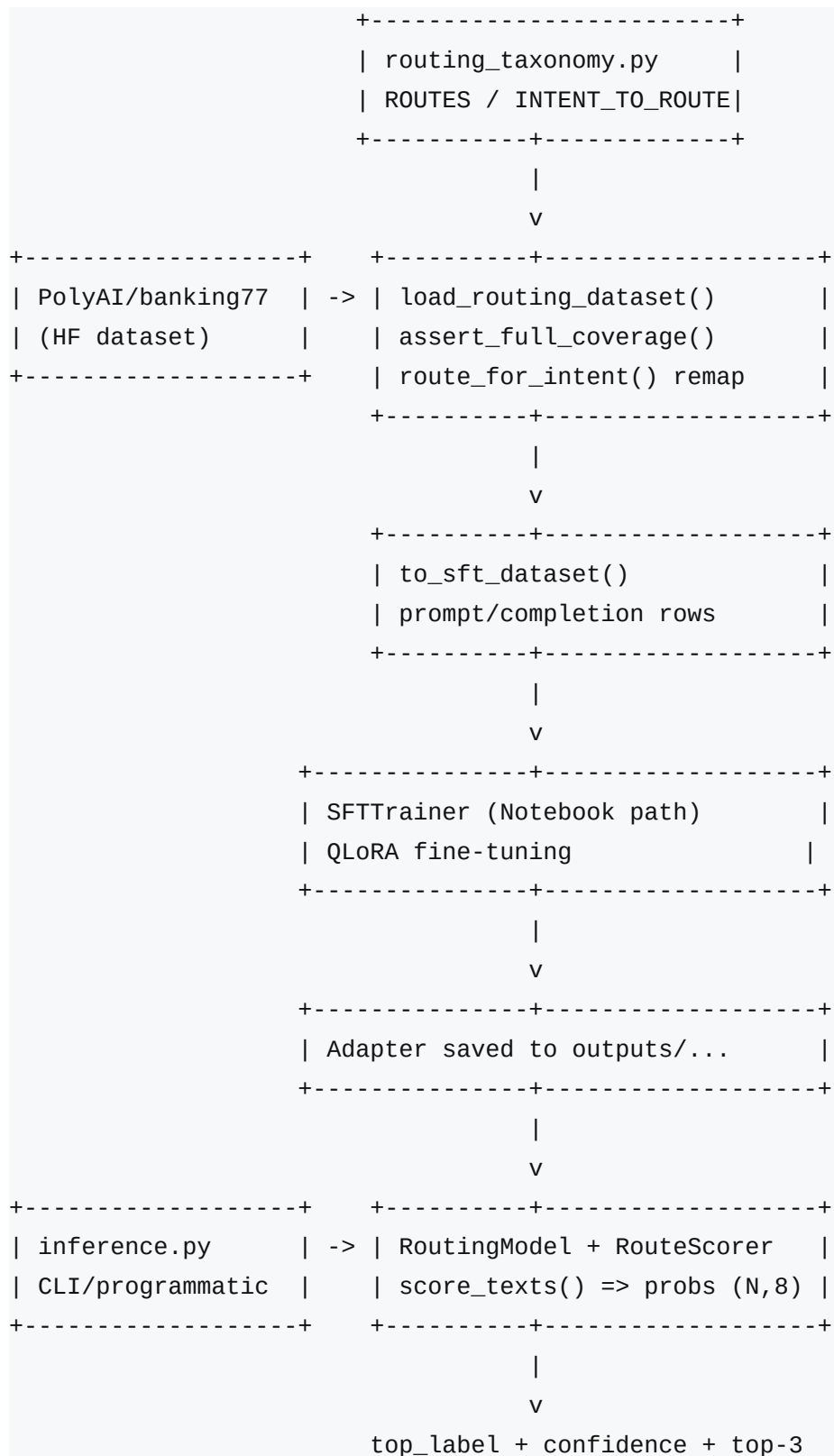
- Data pipeline pattern: Definition: data moves through deterministic transformation stages. In this repo: `load_routing_dataset()` remaps labels, stratifies splits, and returns a `DatasetDict` in [routing_pipeline.py](#).
- Hybrid functional + object-oriented style: Definition: pure-ish helper functions for transformations plus classes for stateful runtime components. In this repo: functions like `build_messages()`, `to_sft_dataset()`, `routing_metrics()` plus classes `RouteScorer` and `RoutingModel`.
- Configuration object pattern: Definition: central dataclass for hyperparameters and paths. In this repo: `RoutingConfig` and shared singleton `CONFIG` in [routing_pipeline.py](#).
- Parameter-Efficient Fine-Tuning (PEFT) with LoRA/QLoRA: Definition: update only low-rank adapter weights while keeping base model frozen, with 4-bit quantized base for memory efficiency. In this repo: LoRA settings in `RoutingConfig` and training setup in generated notebook content from [make_notebook.py](#).
- Constrained label scoring for generative models: Definition: score a fixed set of candidate labels as completions instead of free-form generation. In this repo: `RouteScorer.score_texts()` computes per-route probabilities over `ROUTES`.
- Calibration-aware evaluation: Definition: evaluate whether confidence values match empirical correctness. In this repo: `expected_calibration_error()` in [inference.py](#).

1.3 Architecture overview

Core architecture is shared-library-first: - `routing_taxonomy.py` defines domain taxonomy and label mapping integrity checks. - `routing_pipeline.py` defines config, dataset loading/remapping, prompt building, label scoring, and core metrics. - `make_notebook.py` generates the notebook that orchestrates end-to-end training/evaluation. - `inference.py` reuses shared components for CLI/programmatic routing.

Main runtime paths: - Training/evaluation path: notebook generated from `make_notebook.py`. - Inference path: `inference.py` CLI or `RoutingModel` programmatic API.

ASCII architecture diagram:



Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	
README.md	Project purpose, setup commands, workflow expectations	N/A	Training
.python-version	Pins Python version for environment tools	N/A	3.12.1
pyproject.toml	Dependency and uv project configuration	N/A	requirements [tool.
uv.lock	Reproducible dependency lock for uv	N/A	Locked p
routing_taxonomy.py	Domain taxonomy and intent-to-route mapping integrity	route_for_intent(), assert_full_coverage()	ROUTES, INTENT.
routing_pipeline.py	Shared training/inference plumbing	RoutingConfig, load_routing_dataset(), build_messages(), to_sft_dataset(), RouteScorer.score_texts(), routing_metrics()	CONFIG, path field
inference.py	Reusable inference API and CLI entrypoint	RouteResult, RoutingModel, expected_calibration_error(), main()	_DEMOS, CONFIG
make_notebook.py	Programmatically generates the main notebook deliverable	md(), code() helper builders; notebook assembly logic	OUT = "lora_
lora_dispute_ticket_routing.ipynb	Main executable study artifact (training + evaluation)	Notebook cells import shared modules	Uses COM Routes
_granite_smoke.py	Quick smoke	Inlined flow using SFTTrainer,	MID, tiny

File/Directory Path	Primary Responsibility	Key Classes/Functions	
	script for Granite backbone + LoRA target modules	RouteScorer	target.
<code>_phi_smoke.py</code>	Quick smoke script for Phi backbone and fused target modules	Inlined flow using SFTTrainer, RouteScorer	MID, tiny gate_u
<code>figures/</code>	Stored generated analysis charts	N/A	Files like confus backbo
<code>outputs/</code> (git-ignored except <code>.gitkeep</code>)	Trained adapter/checkpoint output location	N/A	output inference

Module 3: Core Execution Flows

Flow A: Taxonomy + dataset creation pipeline

Entrypoint usage: - Imported by notebook path and smoke scripts. - Core function:
`load_routing_dataset(cfg=CONFIG)`.

Step-by-step: 1. Load Banking77 with `load_dataset("PolyAI/banking77")`. 2. Extract live intent names from `raw["train"].features["label"].names`. 3. Validate mapping sync using `assert_full_coverage(intent_names)`. 4. Remap each original label index to route string via `route_for_intent(intent_name)`. 5. Add integer route labels via `ROUTE2ID[route]`. 6. Create stratified train, validation, and test subsets using `stratified_sample()`. 7. Return a `DatasetDict(train=..., validation=..., test=...)`.

Key input/output shapes: - Input dataset rows before remap: at least `text: str, label: int` (Banking77).
- Output rows after remap: - `text: str` - `route: str` (one of 8 values in `ROUTES`) - `label: int` (0..7 from `ROUTE2ID`)

Short code fragment:

```
routes = [route_for_intent(intent_names[i]) for i in batch["label"]]
return {"route": routes, "label": [ROUTE2ID[r] for r in routes]}
```

Flow B: Prompt construction + SFT dataset formatting

Core functions: - `build_messages(text: str) -> list[dict]` - `to_sft_dataset(ds, tokenizer)`

Step-by-step: 1. Build one system message containing routing rubric (`SYSTEM_PROMPT`). 2. Build one user

message containing complaint text: `Customer message: "...`". 3. Render chat messages into a single text prompt using tokenizer chat template. 4. Emit SFT rows with two fields: prompt string + completion route name.

Exact message structure from `build_messages()`:

```
[
  {"role": "system", "content": SYSTEM_PROMPT},
  {"role": "user", "content": f'Customer message: "{text.strip()}"'},
]
```

Exact SFT row structure from `to_sft_dataset()`:

```
{
  "prompt": "<chat-template-rendered-string>",
  "completion": "<route-name>"
}
```

Flow C: Constrained scoring inference (shared by baseline and fine-tuned)

Core class/method: - `RouteScorer.score_texts(texts: list[str]) -> np.ndarray`

Step-by-step: 1. Pre-tokenize all route names once in `RouteScorer.__init__`. 2. For each input text, create prompt token IDs with `_prompt_ids()`. 3. Build all candidate pairs (`prompt_ids`, `route_ids`) for 8 routes. 4. Batch forward passes over combined tokens. 5. Compute token log-probabilities only over completion segment. 6. Length-normalize log-probability for each candidate route. 7. Softmax over 8 routes to produce normalized probabilities.

Exact output contract: - Return array shape: `(len(texts), len(ROUTES))`. - Each row sums to approximately `1.0` after softmax. - Column index `i` corresponds to `ROUTES[i]`.

Short code fragment:

```
scores = np.full((len(texts), len(self.routes)), -1e9, dtype=np.float64)
...
s = scores - scores.max(axis=1, keepdims=True)
p = np.exp(s)
p /= p.sum(axis=1, keepdims=True)
return p
```

Flow D: Metrics and calibration

Core functions: - `routing_metrics(probs, labels, k=3) -> dict` -
`expected_calibration_error(probs, labels, n_bins=10) -> float`

`routing_metrics()` output keys: - `accuracy` - `macro_f1` - `top3_accuracy` (default `k=3`)

`expected_calibration_error()`: - Uses top-1 confidence bins and weighted confidence-vs-accuracy gap. - Returns a scalar float (`0.0` is perfectly calibrated).

Flow E: Runtime inference API + CLI

Entrypoint: - `if __name__ == "__main__": main()` in `inference.py`.

Step-by-step: 1. Parse CLI args: `--text` and `--base-only`. 2. Instantiate

`RoutingModel(use_base_only=args.base_only)`. 3. Inside `RoutingModel.__init__`: -

Load tokenizer and set `pad_token` if missing. - Load quantized base model (4-bit NF4). - If adapter path exists and not base-only, load adapter via `PeftModel.from_pretrained`. - Build `RouteScorer` over loaded model. 4. Route input(s) with `route_batch()`. 5. Convert probabilities to `RouteResult` objects. 6. Print top label, confidence, and top-3 list.

`RouteResult` exact shape: - `top_label: str` - `confidence: float` - `top_3: list[tuple[str, float]]`

Short code fragment:

```
order = np.argsort(-p)
top3 = [(ID2ROUTE[int(i)], float(p[i])) for i in order[:3]]
RouteResult(ID2ROUTE[int(order[0])], float(p[order[0]]), top3)
```

Flow F: Notebook generation path

Entrypoint: - `make_notebook.py` writes `lora_dispute_ticket_routing.ipynb`.

Step-by-step: 1. Build a `cells` list using helper builders `md()` and `code()`. 2. Append tutorial/training/evaluation cells (dataset, baseline, QLoRA, metrics, confusion, calibration, backbone comparisons). 3. Assemble notebook metadata (`ipykernel`, `python 3.12.10`). 4. Write notebook to `OUT = "lora_dispute_ticket_routing.ipynb"`.

Why this matters for learners: - It is a reproducible “source of truth” for notebook content. - Shared logic still lives in importable modules (`routing_pipeline.py`, `routing_taxonomy.py`, `inference.py`) to avoid drift.

Module 4: Setup & Run Guide

This section documents the expected setup from repository files only.

4.1 Prerequisites

- OS with NVIDIA GPU support.
- CUDA-compatible driver (project expects GPU path for training notebook).
- `uv` installed.
- Python `3.12.10` (`.python-version` and `pyproject.toml` align).

4.2 Dependencies

Defined in `pyproject.toml` and locked in `uv.lock`. Key packages include: - `torch`, `transformers`, `peft`, `bitsandbytes`, `trl`, `datasets`, `accelerate` - `scikit-learn`, `numpy`, `pandas`, `matplotlib`, `seaborn` - `jupyter`, `ipykernel`, `ipywidgets`, `nbformat`

4.3 Environment variables and config files

Observed environment variables in project code: -

`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` (set in generated notebook for memory fragmentation mitigation). - `TOKENIZERS_PARALLELISM=False` (set in generated notebook).

`.env` keys: - No `.env` file or required project-specific secret keys are referenced in repository Python files.

Core config sources: - `RoutingConfig` (`routing_pipeline.py`) for model ID, split sizes,

LoRA/training knobs, and output paths. - `pyproject.toml` for package/runtime constraints.

4.4 Typical command sequence

From repository docs/workflow:

```
# install env and dependencies
uv sync

# regenerate notebook from source generator (optional if notebook already
committed)
uv run python make_notebook.py

# open notebook for end-to-end training/evaluation
uv run jupyter lab lora_dispute_ticket_routing.ipynb
# or: uv run jupyter notebook lora_dispute_ticket_routing.ipynb

# run inference with saved adapter
uv run python inference.py --text "I was charged twice for one purchase"

# run built-in demo inference samples
uv run python inference.py

# force zero-shot base-only inference
uv run python inference.py --base-only

# taxonomy summary sanity check
uv run python routing_taxonomy.py
```

4.5 External resources and first-run behavior

Expected network pulls during first execution: - Model weights/tokenizer from Hugging Face Hub (`CONFIG.model_id`, default `Qwen/Qwen2.5-1.5B-Instruct`). - Dataset `PolyAI/banking77`.

4.6 Migrations/seeding

- No SQL/database migration layer exists in this repository.
- No seeding scripts exist beyond dataset loading/splitting logic in `load_routing_dataset()`.

4.7 PDF export (handbook)

The handbook is markdown-first and can be exported with Pandoc:

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Optional ToC and section numbering:

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf --toc --number-sections
```

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan

Recommended order for a new learner:

1. Read `README.md` for project objective, metrics, and operational framing.
2. Read `routing_taxonomy.py` to internalize labels, mapping, and drift-guard logic.
3. Read `routing_pipeline.py` end-to-end; treat this as the architecture core.
4. Read `inference.py` to understand runtime serving and output contracts.
5. Read `make_notebook.py` to understand orchestration, training configuration, and analysis workflow.
6. Open `lora_dispute_ticket_routing.ipynb` to connect generated notebook narrative with module code.
7. Inspect `_granite_smoke.py` and `_phi_smoke.py` to understand backbone-specific LoRA target differences.
8. Review `figures/` artifacts for evaluation interpretation habits.

5.2 Practice exercises

1. Taxonomy integrity walkthrough: Explain exactly how `assert_full_coverage()` detects drift and what `missing` vs `extra` represent.
2. Route mapping comprehension: Pick any three intents from `INTENT_TO_ROUTE` and explain why they map to their chosen route queues using `ROUTE_DESCRIPTIONS`.
3. Data split reasoning: Trace how `load_routing_dataset()` prevents validation leakage into training and where this is implemented.
4. Prompt contract check: Write down the exact two-message structure produced by `build_messages()` and explain why completion-only training depends on this stable format.
5. Scoring internals: In `RouteScorer.score_texts()`, explain why log-likelihood is length-normalized and why softmax happens over routes, not over tokens.
6. API contract test (by reading only): From `RoutingModel.route_batch()`, specify the exact return type and the semantic meaning of each field.
7. Metrics interpretation: Explain how `routing_metrics()` computes `top3_accuracy` and why this can be high even when `accuracy` is lower.
8. Backbone adaptation analysis: Compare `_granite_smoke.py` vs `_phi_smoke.py` and identify

the concrete LoRA `target_modules` differences and likely architectural reason.

9. Configuration tracing: List all fields of `RoutingConfig` that directly affect memory pressure and describe each field's role.
10. Notebook generation architecture: Explain why this repo uses `make_notebook.py` instead of hand-editing notebook JSON and how that affects maintainability.

5.3 Solution outlines

1. Taxonomy integrity walkthrough (outline): `live = {dataset_intents}` and `mapped = set(INTENT_TO_ROUTE)`; `missing = live - mapped` catches unmapped dataset labels; `extra = mapped - live` catches stale local mappings; assertion fails if either set is non-empty.
2. Route mapping comprehension (outline): Example: `card_swallowed` -> ATM & Cash; `verify_my_identity` -> Identity & Verification; `transfer_not_received_by_recipient` -> Payments & Transfers. Justification is read from each queue's text in `ROUTE_DESCRIPTIONS`.
3. Data split reasoning (outline): Validation is sampled from `train_full`, then `val_ids = set(val["text"])`; training pool is filtered with `train_full.filter(lambda r: r["text"] not in val_ids)` before train sampling.
4. Prompt contract check (outline): The prompt always has one system rubric plus one user complaint message; `to_sft_dataset()` renders this template and stores route name in `completion`, enabling `completion_only_loss=True` to train only the label segment.
5. Scoring internals (outline): Length normalization (`sum_lp / n_tok`) avoids bias toward short route strings; softmax over 8 route candidates converts relative route scores into a probability distribution used for top-1/top-3/confidence.
6. API contract test (outline): `route_batch(texts: list[str]) -> list[RouteResult]`; each result has `top_label` (best route), `confidence` (probability of top route), `top_3` (sorted route-probability pairs for first 3 ranks).
7. Metrics interpretation (outline): `topk = np.argsort(-probs, axis=1)[: , :k]` and hit if true label appears in top-k row. A model can miss top-1 often but still include true label in top-3.
8. Backbone adaptation analysis (outline): Granite uses dense targets (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`); Phi uses fused targets (`qkv_proj`, `o_proj`, `gate_up_proj`, `down_proj`) because projection layer naming/structure differs.
9. Configuration tracing (outline): Memory-sensitive fields: `max_length`, `per_device_batch_size`, `grad_accum`, `eval_batch_size`, `lora_r`, `lora_target_modules`, and quantization choice implied in model loading (`BitsAndBytesConfig`).
10. Notebook generation architecture (outline): `make_notebook.py` keeps notebook content versionable as plain Python; shared logic is imported from modules, reducing notebook drift and easing code review.

Understanding Checklist

Use this self-check before claiming mastery:

- Can you explain why this project uses constrained route scoring instead of free-text generation?
- Can you describe the full data path from Banking77 label IDs to 8-route integer labels?
- Can you reconstruct the exact prompt/completion schema used for SFT training?
- Can you explain `RouteScorer.score_texts()` from pair construction through softmax output?
- Can you state what each field in `RouteResult` means and how it is computed?
- Can you explain the difference between `accuracy`, `macro_f1`, `top3_accuracy`, and `ECE` in this repository?
- Can you identify where adapter loading happens and how base-only mode is toggled?
- Can you list the main memory-control choices (`4-bit`, batch sizes, gradient checkpointing, max length) and their locations in code?
- Can you explain why `assert_full_coverage()` is a critical production safety guard?
- Can you outline the end-to-end notebook flow from baseline through fine-tuning to calibration plots?